

Smart Lender: AI-Powered Loan Eligibility Prediction System

Project Description

Description

Smart Lender is an Artificial Intelligence and Machine Learning-based web application designed to predict the loan eligibility of applicants. The system helps banks and financial institutions make faster, smarter, and more accurate loan approval decisions by analyzing applicant information using a trained Machine Learning model.

The application collects details such as Gender, Marital Status, Education, Employment Status, Applicant Income, Co-applicant Income, Loan Amount, Loan Amount Term, Credit History, and Property Area. These details are processed using a Random Forest Classification model, which predicts whether the applicant is eligible for a loan. The system is developed using Flask, HTML, CSS, Bootstrap, JavaScript, SQLite, and Python. It provides secure user authentication, instant prediction results, and stores prediction history for future reference. Smart Lender minimizes manual work, reduces loan approval time, and improves decision-making accuracy.

Scenarios

Scenario 1 – Loan Approval

A bank employee enters the details of a salaried applicant with a good credit history. The system predicts that the applicant is eligible for the loan within seconds.

Scenario 2 – Loan Rejection

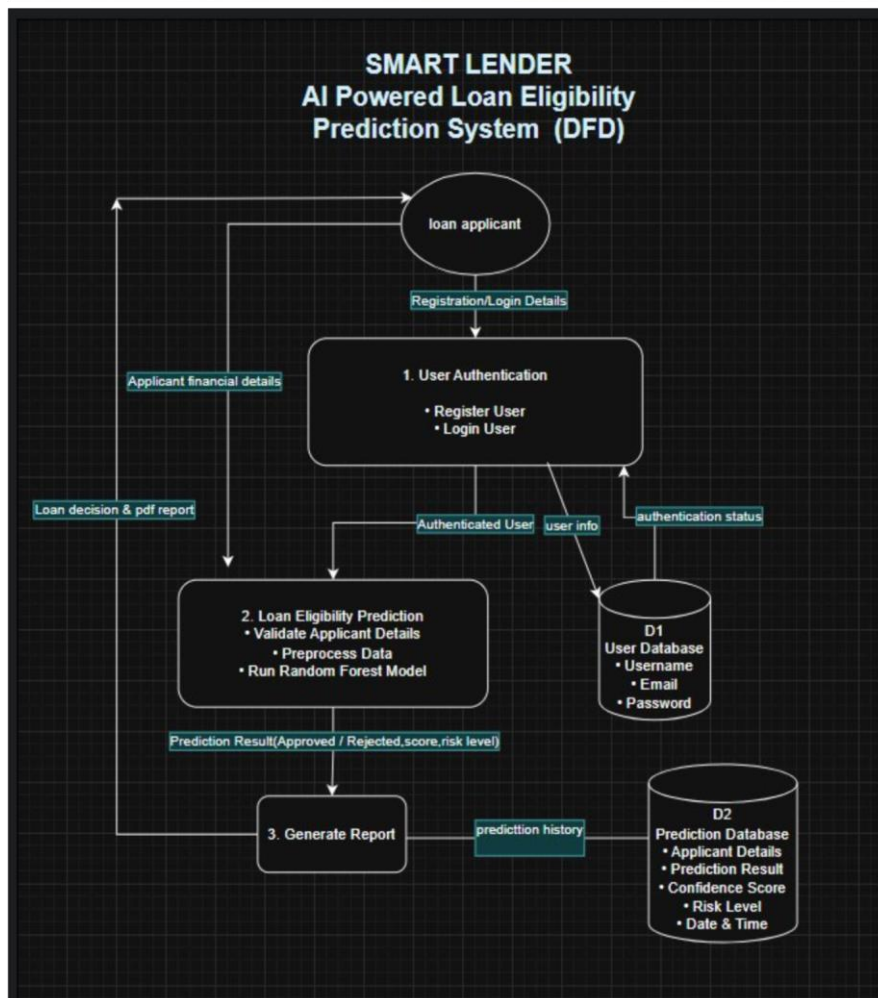
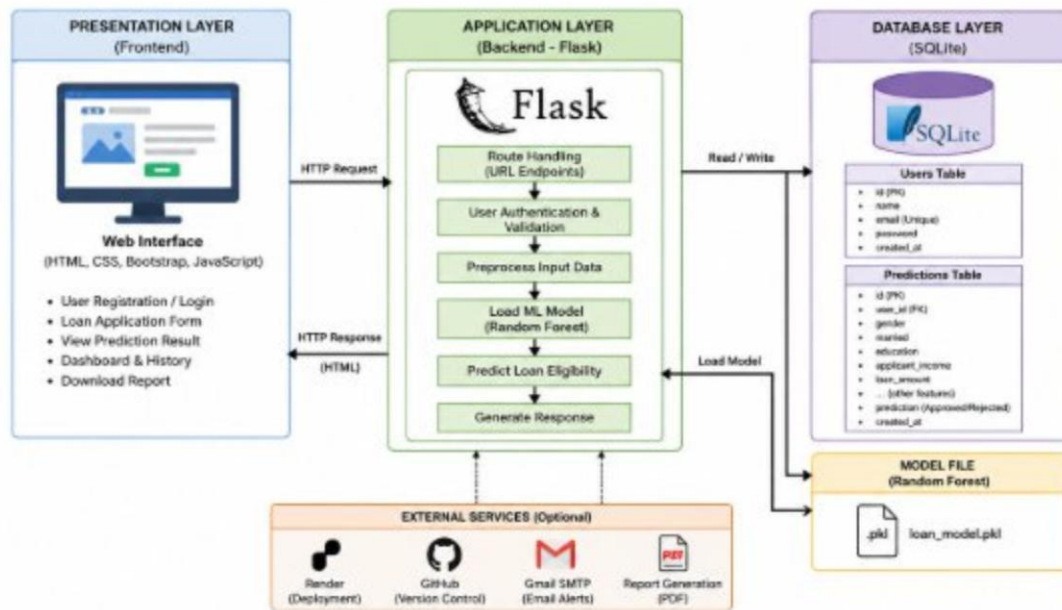
An applicant with low income and poor credit history submits a loan request. The Random Forest model predicts that the loan should be rejected, helping reduce financial risk.

Scenario 3 – Faster Decision Making

A financial institution receives hundreds of loan applications every day. Smart Lender automates the loan evaluation process, reducing manual effort and improving processing speed.

Technical Architecture:

Technical Architecture:



Description

Smart Lender follows a three-tier architecture consisting of the Presentation Layer, Application Layer, and Database Layer.

The Presentation Layer provides a responsive user interface developed using HTML, CSS, Bootstrap, and JavaScript. Users can register, log in, and submit loan application details through an interactive web interface.

The Application Layer is developed using the Flask framework. It handles user authentication, validates loan application data, loads the trained Random Forest model, predicts loan eligibility, and generates the final prediction.

The Database Layer uses SQLite to securely store user credentials and loan prediction records. All prediction results are saved for future reference and analysis.

This architecture ensures scalability, security, faster response time, and efficient loan processing.

Pre-requisites

- Python 3.10 or above
- Flask Framework
- Machine Learning using Scikit-Learn
- Pandas
- NumPy
- Pickle
- HTML
- CSS
- Bootstrap
- JavaScript
- SQLite Database
- VS Code
- Git & GitHub
- Render Deployment

•

Project Workflow

Milestone 1: Model Selection and Architecture Activity 1.1: Dataset Collection

Collect the loan prediction dataset containing applicant information and loan status.

1	Loan_ID	Gender	Married	Dependent	Education	Self_Emplo	ApplicantLi	Coapplicant	LoanAmou	Loan_Amc	Credit_His	Property_	Loan_Status
2	LP001002	Male	No	0	Graduate	No	5849	0		360	1	Urban	Y
3	LP001003	Male	Yes	1	Graduate	No	4583	1508	128	360	1	Rural	N
4	LP001005	Male	Yes	0	Graduate	Yes	3000	0	66	360	1	Urban	Y
5	LP001006	Male	Yes	0	Not Gradu	No	2583	2358	120	360	1	Urban	Y
6	LP001008	Male	No	0	Graduate	No	6000	0	141	360	1	Urban	Y
7	LP001011	Male	Yes	2	Graduate	Yes	5417	4196	267	360	1	Urban	Y
8	LP001013	Male	Yes	0	Not Gradu	No	2333	1516	95	360	1	Urban	Y
9	LP001014	Male	Yes	3+	Graduate	No	3036	2504	158	360	0	Semiurban	N
10	LP001018	Male	Yes	2	Graduate	No	4006	1526	168	360	1	Urban	Y
11	LP001020	Male	Yes	1	Graduate	No	12841	10968	349	360	1	Semiurban	N
12	LP001024	Male	Yes	2	Graduate	No	3200	700	70	360	1	Urban	Y
13	LP001027	Male	Yes	2	Graduate		2500	1840	109	360	1	Urban	Y
14	LP001028	Male	Yes	2	Graduate	No	3073	8106	200	360	1	Urban	Y
15	LP001029	Male	No	0	Graduate	No	1853	2840	114	360	1	Rural	N
16	LP001030	Male	Yes	2	Graduate	No	1299	1086	17	120	1	Urban	Y
17	LP001032	Male	No	0	Graduate	No	4950	0	125	360	1	Urban	Y
18	LP001034	Male	No	1	Not Gradu	No	3596	0	100	240		Urban	Y
19	LP001036	Female	No	0	Graduate	No	3510	0	76	360	0	Urban	N
20	LP001038	Male	Yes	0	Not Gradu	No	4887	0	133	360	1	Rural	N
21	LP001041	Male	Yes	0	Graduate		2600	3500	115		1	Urban	Y
22	LP001043	Male	Yes	0	Not Gradu	No	7660	0	104	360	0	Urban	N
23	LP001046	Male	Yes	1	Graduate	No	5955	5625	315	360	1	Urban	Y
24	LP001047	Male	Yes	0	Not Gradu	No	2600	1911	116	360	0	Semiurban	N
25	LP001050		Yes	2	Not Gradu	No	3365	1917	112	360	0	Rural	N
26	LP001052	Male	Yes	1	Graduate		3717	2925	151	360		Semiurban	N
27	LP001066	Male	Yes	0	Graduate	Yes	9560	0	191	360	1	Semiurban	Y
28	LP001068	Male	Yes	0	Graduate	No	2799	2253	122	360	1	Semiurban	Y
29	LP001073	Male	Yes	2	Not Gradu	No	4226	1040	110	360	1	Urban	Y

Activity 1.2: Data Preprocessing

Handle missing values, encode categorical variables, normalize data, and prepare the dataset for model training.

```
data.isnull().sum()
```

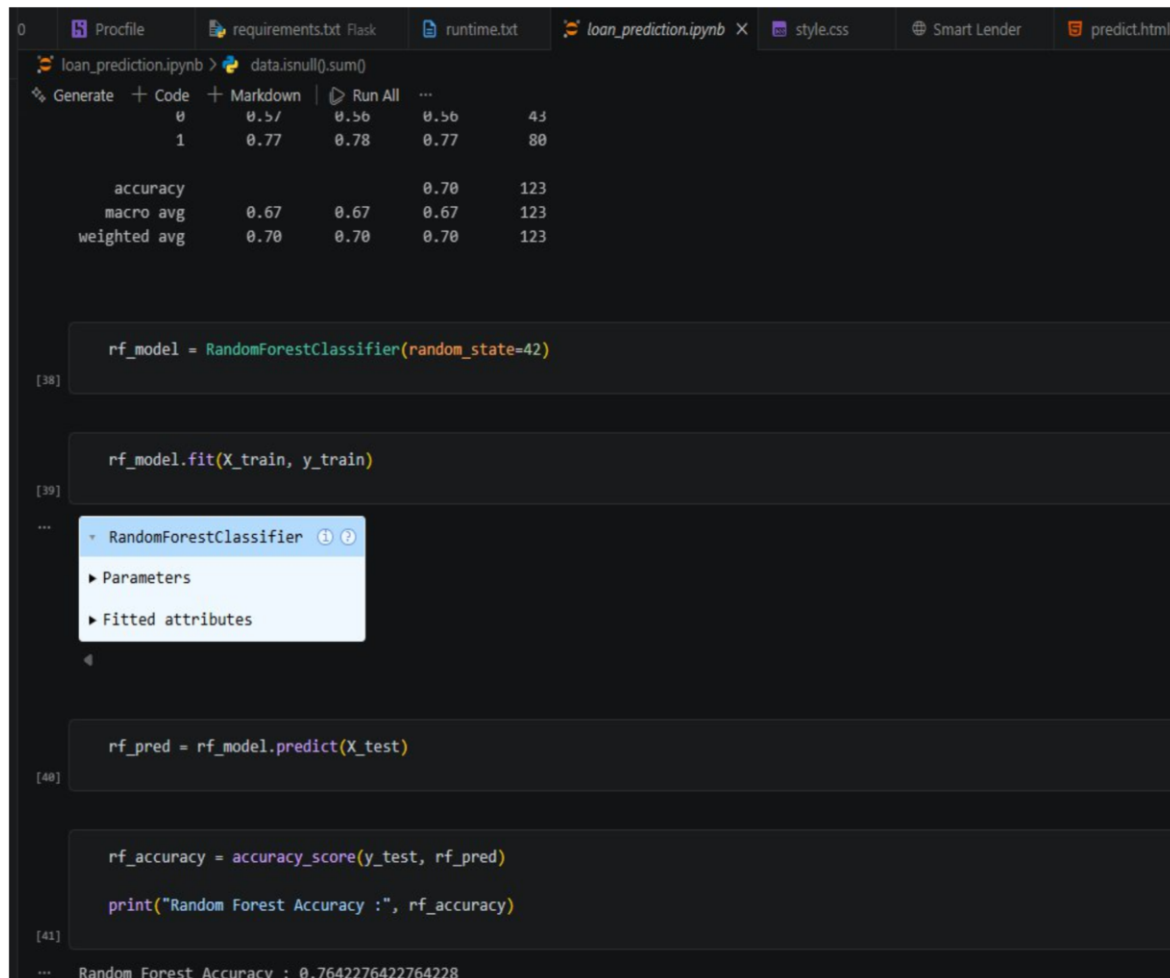
Loan_ID	0
Gender	13
Married	3
Dependents	15
Education	0
Self_Employed	32
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	22
Loan_Amount_Term	14
Credit_History	50
Property_Area	0
Loan_Status	0
dtype:	int64


```
data.describe()
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	614.000000	614.000000	592.000000	600.000000	564.000000
mean	5403.459283	1621.245798	146.412162	342.000000	0.842199
std	6109.041673	2926.248369	85.587325	65.12041	0.364878
min	150.000000	0.000000	9.000000	12.000000	0.000000
25%	2877.500000	0.000000	100.000000	360.000000	1.000000
50%	3812.500000	1188.500000	128.000000	360.000000	1.000000
75%	5795.000000	2297.250000	168.000000	360.000000	1.000000

Activity 1.3: Model Training

Train multiple machine learning models such as Decision Tree, KNN, Random Forest, and XGBoost.



```
loan_prediction.ipynb > data.isnull().sum()
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	614.000000	614.000000	592.000000	600.000000	564.000000
mean	5403.459283	1621.245798	146.412162	342.000000	0.842199
std	6109.041673	2926.248369	85.587325	65.12041	0.364878
min	150.000000	0.000000	9.000000	12.000000	0.000000
25%	2877.500000	0.000000	100.000000	360.000000	1.000000
50%	3812.500000	1188.500000	128.000000	360.000000	1.000000
75%	5795.000000	2297.250000	168.000000	360.000000	1.000000


```
rf_model = RandomForestClassifier(random_state=42)
```

```
rf_model.fit(X_train, y_train)
```

```
rf_pred = rf_model.predict(X_test)
```

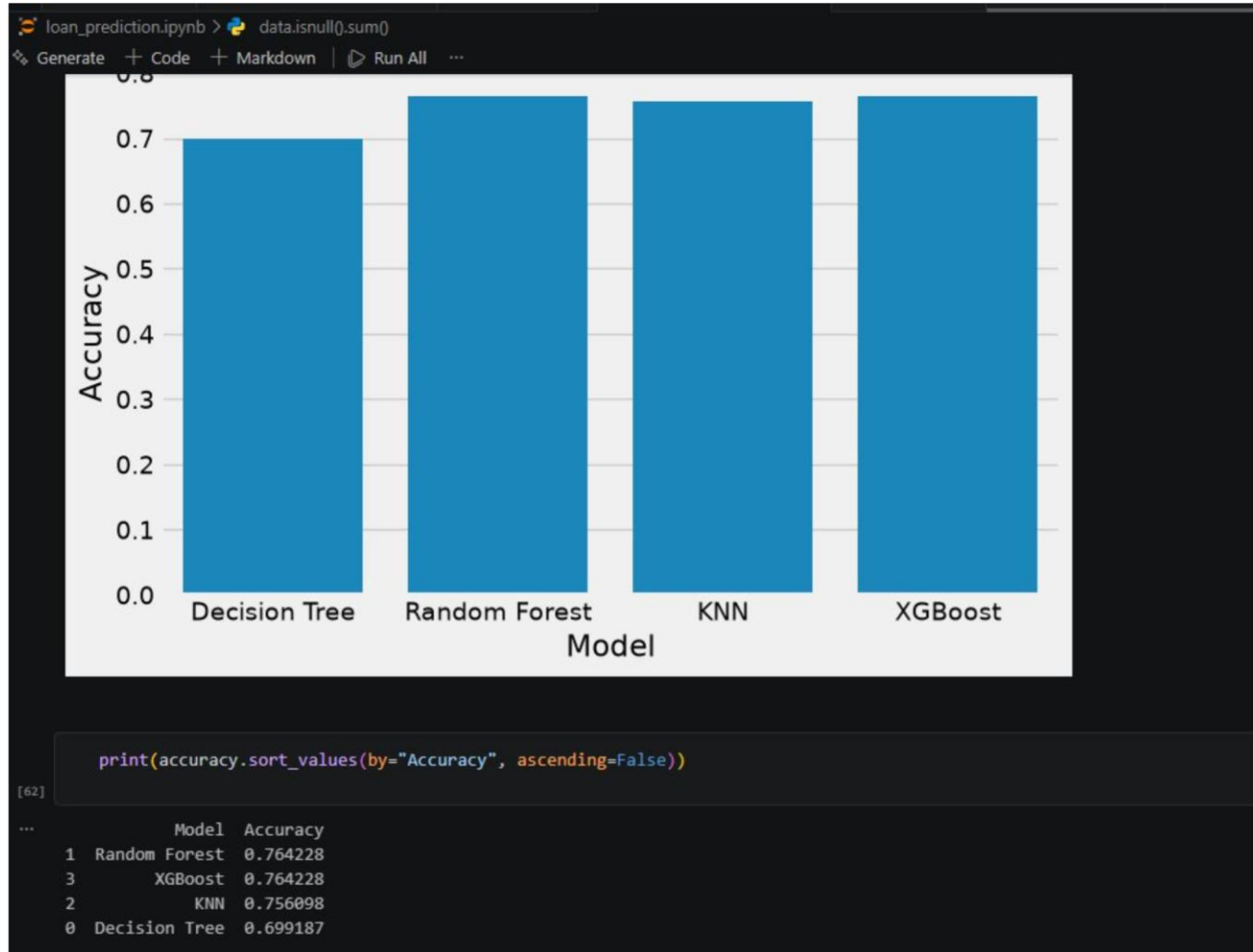
```
rf_accuracy = accuracy_score(y_test, rf_pred)
```

```
print("Random Forest Accuracy :", rf_accuracy)
```

Random Forest Accuracy : 0.7642276422764228

Activity 1.4: Model Selection

Compare model performance using Accuracy Score and select Random Forest as the final prediction model.



Milestone 2: Core Functionalities Development Activity 2.1

Develop User Registration and Login Module.

Activity 2.2

Implement Loan Prediction Module.

Activity 2.3

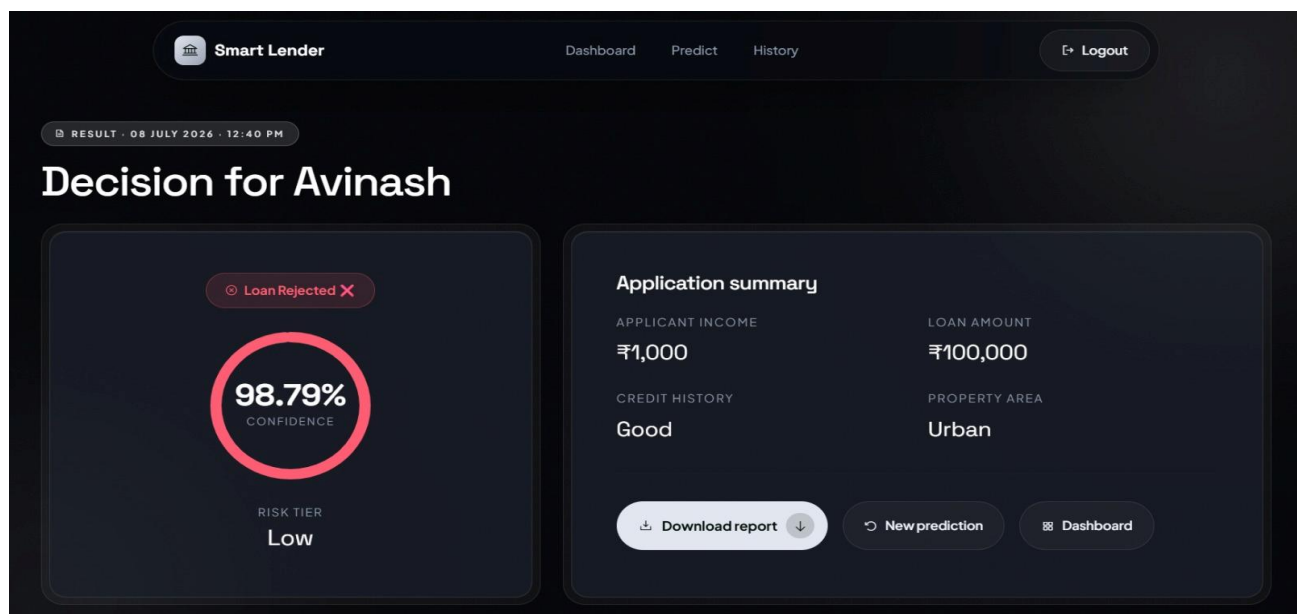
Validate applicant information before prediction.

Activity 2.4

Store prediction results in SQLite Database.

Activity 2.5

Generate prediction report.



Milestone 3: Flask Application Development Activity 3.1

Create Flask routes for

- Home Page
- Login
- Register
- Loan Prediction
- Dashboard
- Logout

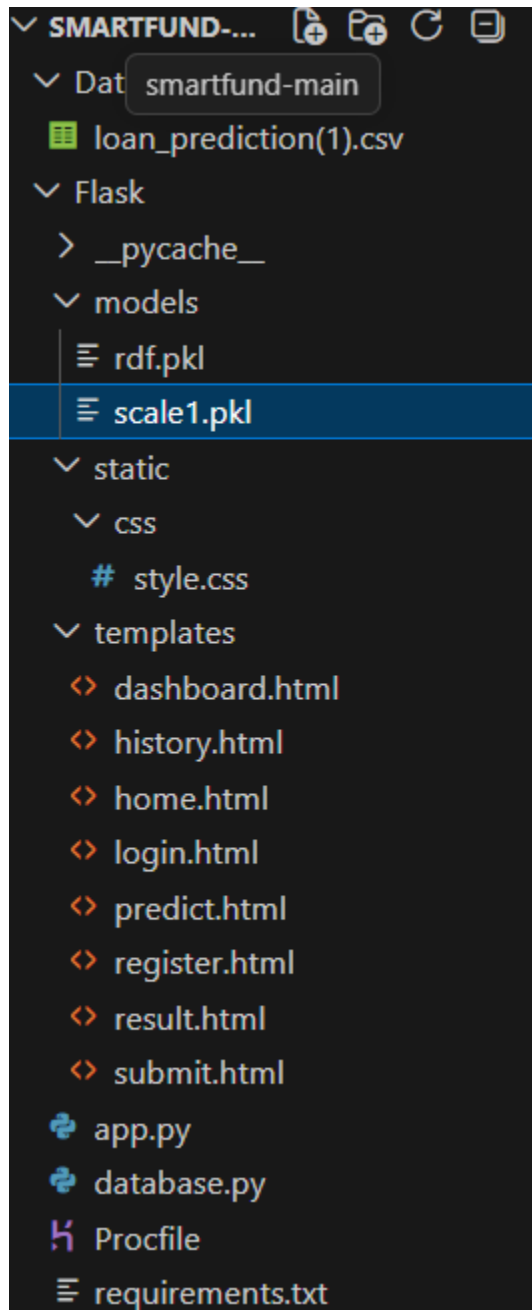

```

app.py x requirements.txt \ .gitignore U Smart Lender | AI Loan Eligibi... history.html Smart Lende
Flask > app.py > download_pdf
9 from reportlab.lib.styles import getSampleStyleSheet
10 from reportlab.pdfbase import pdfmetrics
11 import os
12 from reportlab.lib.enums import TA_CENTER
13 from reportlab.lib.styles import ParagraphStyle
14 app = Flask(__name__)
15 app.secret_key = "smart_lender_secret_key"
16
17 # Load model and scaler
18 model = pickle.load(open("models/rdf.pkl", "rb"))
19 scaler = pickle.load(open("models/scale1.pkl", "rb"))
20
21 create_database()
22 create_users_table()
23
24 @app.route("/register")
25 def register():
26     return render_template("register.html")
27
28
29 @app.route("/login")
30 def login():
31     return render_template("login.html")
32
33
34 @app.route("/logout")
35
36 def logout():
37     session.clear()
38     return redirect("/")
39
40
41
42 @app.route("/")
43 def home():
44     return render_template("home.html")
45 @app.route("/dashboard")
46 def dashboard():

```

Activity 3.2

Load the trained Random Forest model using Pickle.



Activity 3.3

Accept applicant information from HTML form.

Activity 3.4

Predict loan eligibility using the trained model.

Activity 3.5

Store prediction details inside SQLite Database.

app.py | prediction.db | requirements.txt | .gitignore | Smart Lender | AI Loan Eligibility | history.html | Smart Lender | AI Loan Eligibility

Flask > prediction.db

Rows: 10

id	applicant...	gender	married	depende...	education	self_emp...	applicant...	coapplic...
1	pavan	1.0	1.0	1.0	1.0	1.0	2000	30000
2	king	0.0	0.0	2.0	1.0	1.0	10000	20000
3	ravi	1.0	1.0	0.0	1.0	1.0	10000	20000
4	ravi	1.0	1.0	0.0	1.0	1.0	10000	20000
5	siddu	1.0	1.0	1.0	1.0	1.0	10000	200000
6	pavan	1.0	1.0	0.0	1.0	1.0	122000	10000
7	king	0.0	0.0	1.0	0.0	1.0	2000	10000
8	king	0.0	0.0	1.0	0.0	1.0	2000	10000
9	siddu	1.0	1.0	0.0	1.0	1.0	10000	2000
10	sagar	1.0	1.0	2.0	1.0	1.0	50000	100000

Milestone 4: Frontend Development

Activity 4.1


Smart Lender

Bank-grade security

SECURE ACCESS

Welcome back

Sign in to run predictions and view your dashboard.

EMAIL

PASSWORD

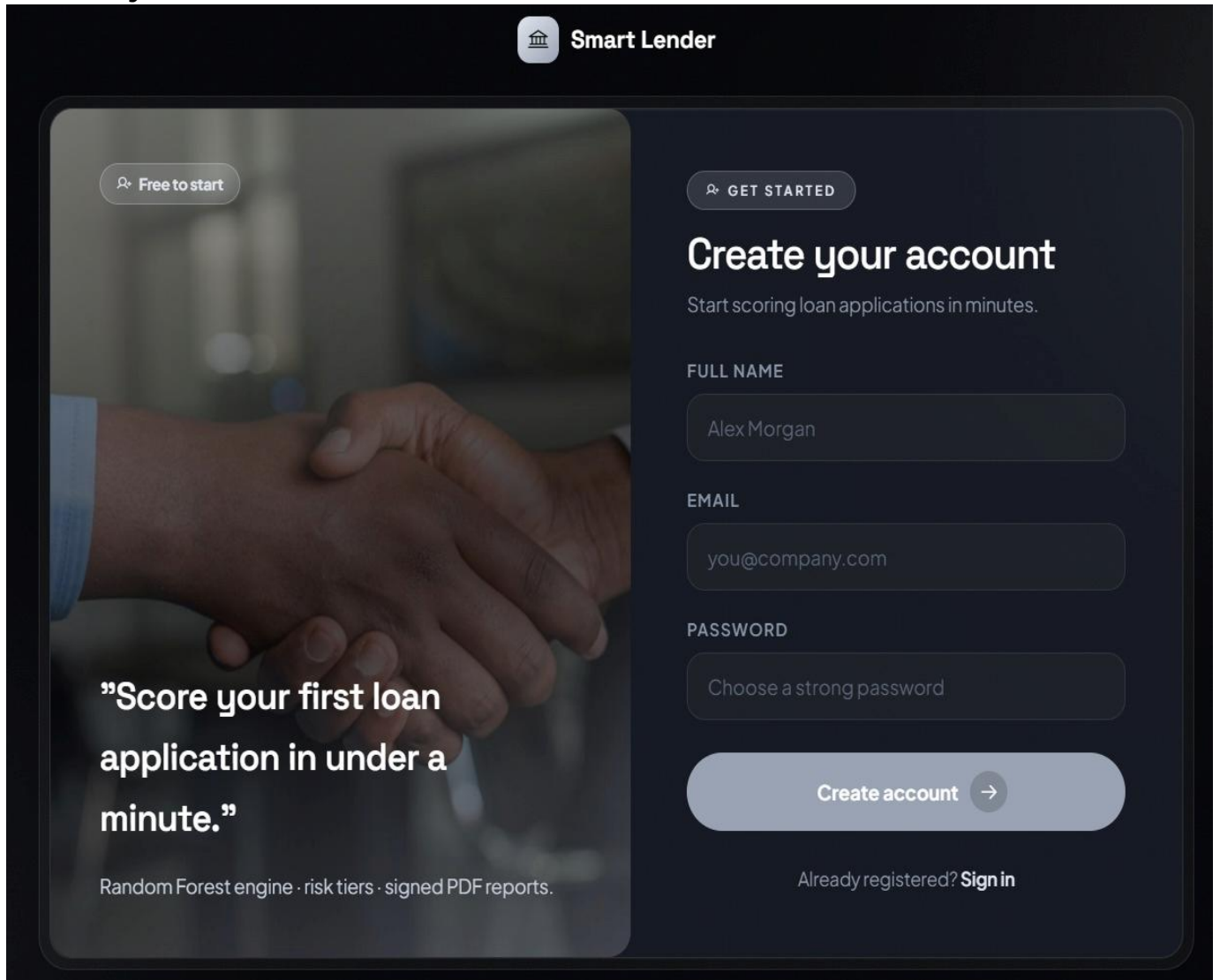
Sign in →

No account? [Create one](#)

"Every loan decision, backed by AI you can defend."

Confidence scores, risk tiers and signed reports — in seconds.

Activity 4.2



The image shows a mobile app interface for 'Smart Lender'. The header features a bank icon and the text 'Smart Lender'. The main content is split into two panels. The left panel has a background image of two hands shaking and contains the text: 'Free to start', 'Score your first loan application in under a minute.', and 'Random Forest engine · risk tiers · signed PDF reports.' The right panel is a 'Create your account' form with the following fields: 'GET STARTED', 'FULL NAME' (with 'Alex Morgan' entered), 'EMAIL' (with 'you@company.com' entered), and 'PASSWORD' (with 'Choose a strong password' entered). At the bottom of the right panel is a 'Create account' button with a right arrow and a link 'Already registered? Sign in'.

Smart Lender

Free to start

GET STARTED

Create your account

Start scoring loan applications in minutes.

FULL NAME

Alex Morgan

EMAIL

you@company.com

PASSWORD

Choose a strong password

Create account →

Already registered? [Sign in](#)

"Score your first loan application in under a minute."

Random Forest engine · risk tiers · signed PDF reports.

Activity 4.3

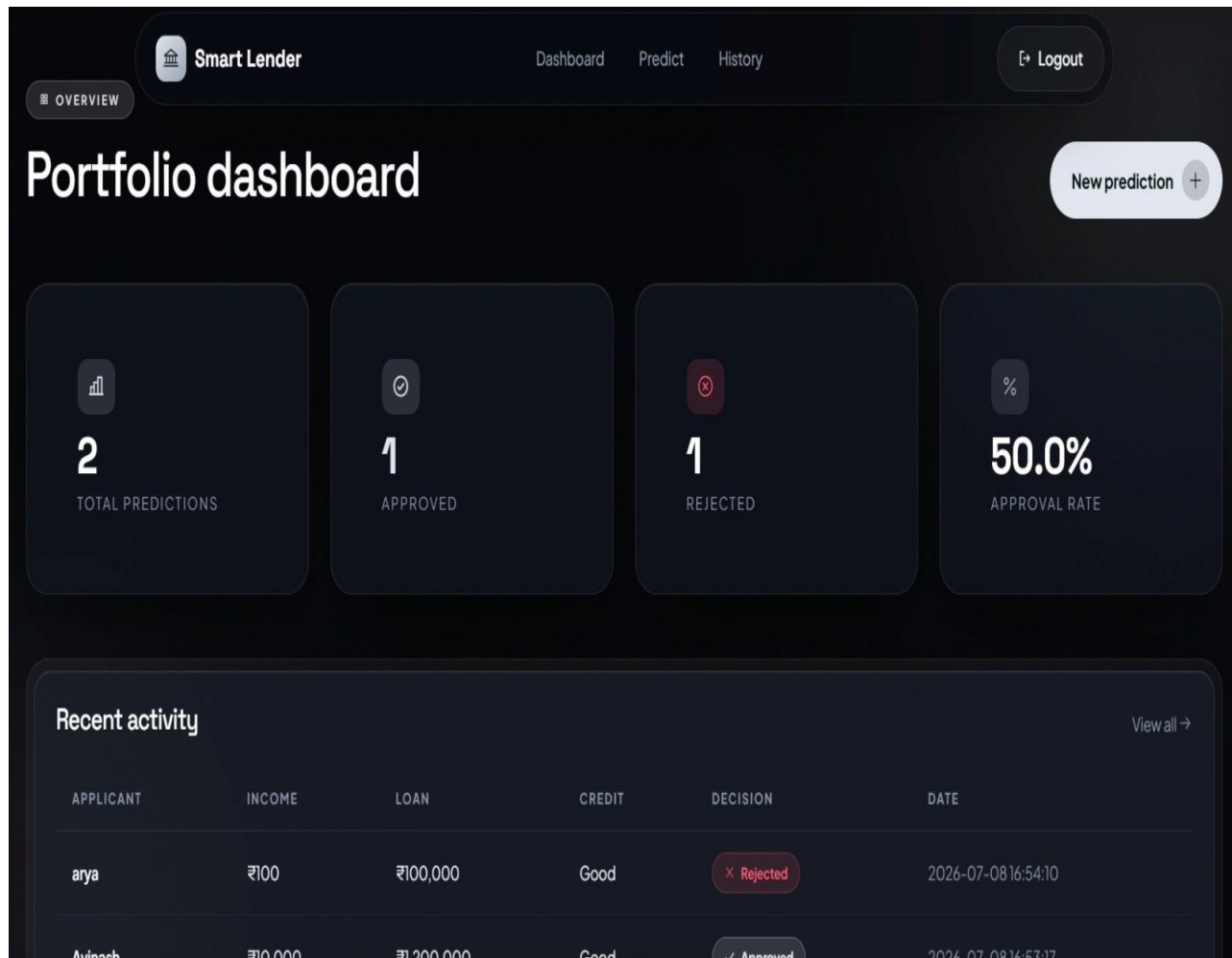
Develop Loan Prediction Form.

Activity 4.4

Display Prediction Results.

Activity 4.5

Develop Dashboard for Prediction History.



Activity 5.1

Create Virtual Environment

```
python -m venv venv
```

Activate Environment

```
venv\Scripts\activate
```

Install Dependencies

```
pip install -r requirements.txt
```

Activity 5.2

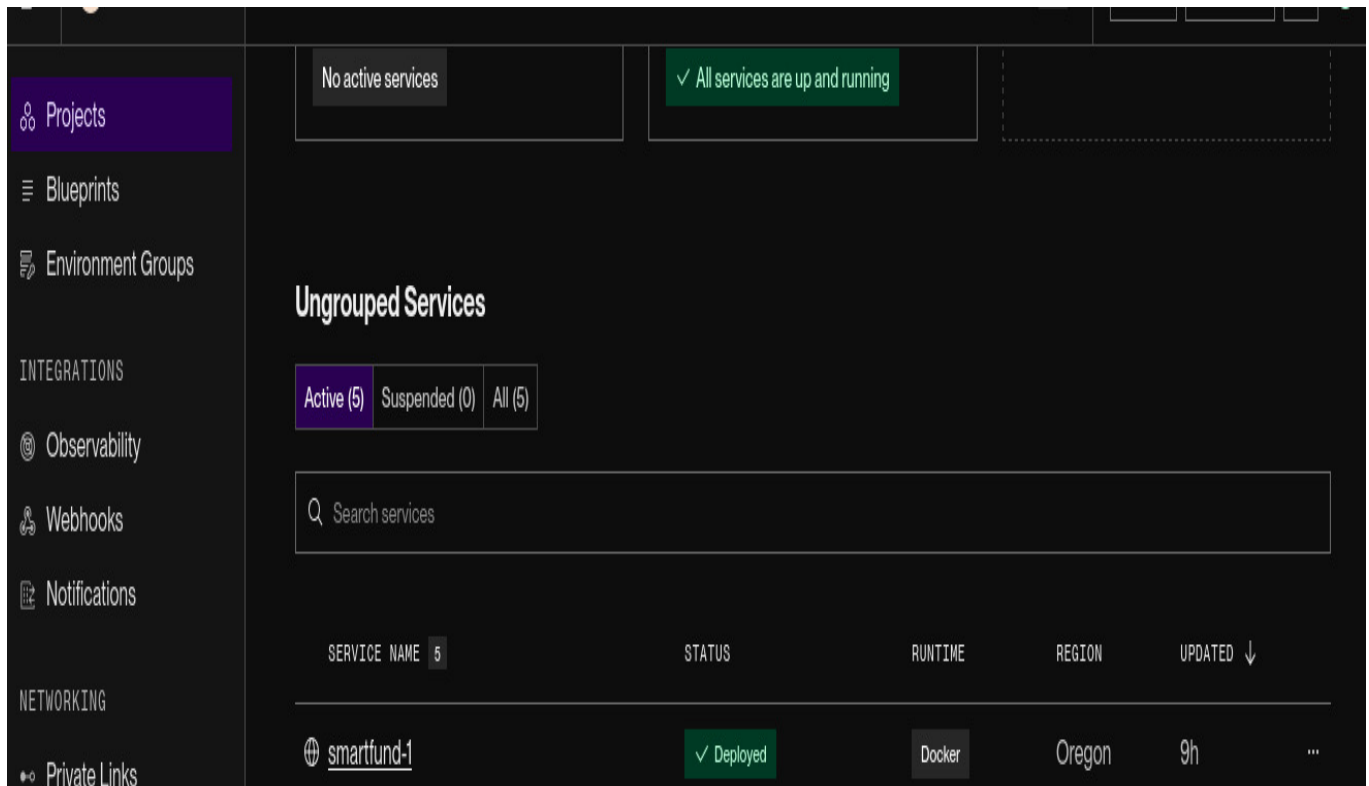
Run Flask Application

```
python app.py
```

Push Source Code to GitHub.

Activity 5.4

Deploy the application on Render.



The screenshot shows a dashboard with a sidebar on the left containing navigation links: Projects, Blueprints, Environment Groups, INTEGRATIONS, Observability, Webhooks, Notifications, NETWORKING, and Private Links. The main content area has a header with two status boxes: 'No active services' and '✓ All services are up and running'. Below this is a section titled 'Ungrouped Services' with filters for 'Active (5)', 'Suspended (0)', and 'All (5)'. A search bar labeled 'Search services' is present. A table lists services with columns: SERVICE NAME, STATUS, RUNTIME, REGION, and UPDATED. One service, 'smartfund-1', is listed with a status of '✓ Deployed', runtime 'Docker', region 'Oregon', and updated '9h'.

SERVICE NAME	STATUS	RUNTIME	REGION	UPDATED
smartfund-1	✓ Deployed	Docker	Oregon	9h

Activity 5.5

Test the deployed application.

<https://smartfund-1.onrender.com/>

Conclusion:

Smart Lender is an AI-powered Loan Eligibility Prediction System that assists banks and financial institutions in making faster and more accurate loan approval decisions. By leveraging the Random Forest Machine Learning algorithm, the application analyzes applicant information and predicts loan eligibility with high accuracy.

The project integrates Machine Learning, Flask, HTML, CSS, Bootstrap, JavaScript, and SQLite into a complete web-based solution. It reduces manual effort, speeds up loan processing, and minimizes financial risk. The system is scalable, user-friendly, and suitable for real-world banking applications.

Future enhancements may include cloud database integration, support for multiple machine learning models, real-time analytics dashboards, document verification, and deployment using advanced cloud platforms to further improve performance and usability.